GITHUB REPO ANALYZER

BYOP — Build Your Own Project

Rewritten in Java | Zero External Dependencies

Student Name	Krish Kumar
Registration No.	24BAI10940
Course	Open Source Software
Language	Java 11+ (zero external dependencies)
GitHub Repo	https://github.com/krish0455/oss-audit-24bai10940
Tool File	byop/GitHubAnalyzer.java
Submission	VITyarthi Portal

"The goal is to turn data into information, and information into insight."
— Carly Fiorina

TABLE OF CONTENTS

01 The Problem
02 Why It Matters
03 My Approach
04 How It Works — Technical Design
05 Java Architecture & Key Decisions
06 The JSON Parser
07 Challenges Faced
08 Sample Output
09 How to Run It
10 What I Learned
11 Conclusion

01 THE PROBLEM

GitHub has over 100 million repositories. As a developer or student, evaluating a repository quickly is a common need — is it actively maintained? What language is it written in? Who are the main contributors? Is it a popular project or just something someone put up and forgot about?

The problem is that getting this information requires clicking through multiple pages on GitHub's website. The main repo page shows stars and forks. The insights tab shows contributor graphs. The commits page shows recent activity. The releases page shows the latest version. You end up with a lot of browser tabs just to understand one repository.

There is no single command-line tool that fetches all of this at once, displays it cleanly in the terminal, and saves it as a report you can refer back to later. That is the gap this project fills.

02 WHY IT MATTERS

This problem matters for several groups of people:

- **Students** — evaluating open-source projects for this course — a quick overview of a repository's health, age, activity, and language breakdown is genuinely useful.
- **Developers** — researching libraries or frameworks before adopting them. How many contributors are actively working on it? Is there a recent release? Are there open issues?
- **Researchers** — comparing repositories across multiple projects to understand trends in the ecosystem. The search history feature allows exactly that kind of ongoing comparison.

The tool also demonstrates something directly relevant to this course: using the GitHub API — itself public open-source infrastructure — to analyse open-source projects. A tool built with open source, for understanding open source.

03 MY APPROACH

The approach was to build a pure Java command-line tool that requires no external dependencies. Everything uses the Java standard library (JDK 11+): java.net.http for HTTP requests, java.nio.file for file I/O, java.time for date formatting, and a hand-rolled recursive-descent JSON parser bundled inside the same file. This was a deliberate choice: the tool should compile and run on any machine with JDK 11 installed — no Maven, no Gradle, no JAR files.

Aspect	Detail
API Used	GitHub REST API v3 (api.github.com) — free, no key required for basic use
Data Fetched	Repo info, languages, contributors, commits, issues, releases
HTTP Client	java.net.http.HttpClient (JDK 11 built-in)
JSON Parsing	Custom recursive-descent parser — zero dependencies
Output	Formatted terminal display + .txt report in reports/ folder
History	Every search logged to search_history.csv with timestamp

Dependencies	None — pure Java 11 standard library only

04 HOW IT WORKS — TECHNICAL DESIGN

The tool is a single Java file: GitHubAnalyzer.java. Here is the flow when you run it:

Step	Method	What It Does
1	`parseRepoUrl()`	Extracts owner and repo name from any GitHub URL format
2	`fetchRepoInfo()`	Calls /repos/{owner}/{repo} — returns stars, forks, dates, license
3	`fetchLanguages()`	Calls /repos/{owner}/{repo}/languages — bytes per language
4	`fetchContributors()`	Calls /contributors endpoint — top 10 by commit count
5	`fetchRecentCommits()`	Calls /commits — last 5 commit messages, authors, dates
6	`fetchOpenIssues()`	Calls /issues — 5 most recent open issues (PRs filtered out)
7	`fetchRelease()`	Calls /releases — latest release tag, date, and notes
8	`analyze()`	Prints formatted output to terminal AND builds report string
9	`saveReport()`	Writes the full report to reports/{owner}_{repo}_{timestamp}.txt
10	`logHistory()`	Appends owner, repo, stars, forks, timestamp to search_history.csv

05 JAVA ARCHITECTURE & KEY DECISIONS

No External Dependencies

In Python, urllib handles HTTP with moderate verbosity. In Java, the JDK 11 HttpClient provides a modern async-capable HTTP API that is equally clean. The bigger challenge was JSON: the JDK has no built-in JSON parser, unlike Python's json module. Rather than pulling in Gson or Jackson, this tool ships a self-contained recursive-descent JSON parser (~120 lines) at the bottom of the file. This preserves the zero-dependency promise exactly.

HttpClient (JDK 11+)

Java's java.net.http.HttpClient replaces Python's urllib. It supports connection timeouts, custom headers, and proper HTTP response code handling. All five GitHub API calls are routed through a single apiGet() method that handles 404, 403, 401, and network errors with clean exit messages.

Type System — Maps and Lists

Python's json.loads() returns dicts and lists naturally. The Java equivalent uses Map<String, Object> and List<Object>. Every API response is parsed into these structures, with typed helper methods (toLong, str, map, list) to safely extract fields without null pointer exceptions.

File I/O and History

Python uses the csv module and open() calls. Java uses java.nio.file.Files.writeString() and BufferedWriter for append-mode CSV logging. The logic is identical — reports are saved to reports/ with a timestamp, and every search is appended to search_history.csv.

Compile & Run

```
# Compile (one time only)
javac GitHubAnalyzer.java
```

```
# Run against any public repo
java GitHubAnalyzer https://github.com/pallets/flask

# Optional: token for higher rate limits
export GITHUB_TOKEN=your_token
java GitHubAnalyzer https://github.com/torvalds/linux
```

06 THE JSON PARSER — ZERO DEPENDENCIES

The most interesting engineering decision in the Java port is the self-contained JSON parser. Since the JDK provides no built-in JSON support, and external libraries were off the table, a recursive-descent parser was written from scratch and bundled at the bottom of the same file.

It handles the full JSON specification needed for GitHub API responses: objects {}, arrays [], strings with escape sequences (\n, \t, \uXXXX), integers, floating-point numbers, booleans, and null values. The parser is a single class (JsonParser) with a position cursor walking the input string character by character.

Parser Architecture

```
static class JsonParser {
  parseValue()    // dispatches: { [ " t f n or number
  parseObject()   // returns Map<String, Object>
  parseArray()    // returns List<Object>
  parseString()   // handles all escape sequences including \uXXXX
  parseNumber()   // returns Long or Double
  skipWhitespace()
}
```

The result: any machine with JDK 11 can compile and run the tool with a single javac command, with no internet access required for setup, no package manager, and no version conflicts. This mirrors the original Python tool's design philosophy exactly.

07 CHALLENGES FACED

Writing a JSON Parser Without Libraries

This was the biggest challenge unique to the Java port. Python's json module handles parsing in one line. Java's JDK has nothing equivalent. Writing a recursive-descent parser from scratch required handling all edge cases: nested objects and arrays, escape sequences inside strings (including Unicode \uXXXX), numbers that could be integers or floats, and whitespace between any tokens. Testing it against real GitHub API responses revealed several edge cases that needed fixing.

GitHub API Rate Limiting

The GitHub API allows 60 requests per hour for unauthenticated users. During development, this limit was hit multiple times while testing. The fix was to add clear error handling for HTTP 403 responses and instructions for setting up a personal access token. With a token, the limit rises to 5,000 requests per hour.

ISO 8601 Date Parsing in Java

GitHub returns dates in ISO 8601 format (e.g. 2023-11-15T09:23:41Z). Python's datetime.fromisoformat() handles this with a minor fix. Java's OffsetDateTime.parse() with DateTimeFormatter.ISO_OFFSET_DATE_TIME handles the Z suffix cleanly, but the relative date calculation (days ago, months ago) required careful arithmetic to match the Python output exactly.

Pull Requests Mixed With Issues

The /issues endpoint returns both issues and pull requests. GitHub includes a pull_request key in PR entries. The Java port filters these out using Java streams: issues.stream().filter(i -> !i.containsKey("pull_request")).limit(5). This was the same fix as the Python version, just expressed in stream syntax.

Repos With No Data

Some repositories have no releases, no open issues, no contributors, or no language data. Each section needed a fallback message rather than throwing a NullPointerException. Testing against several types of repositories — large active ones, small personal ones, archived ones — exposed most of these edge cases.

08 SAMPLE OUTPUT

Below is representative output when run against github.com/pallets/flask:

```
==============================================================
  GITHUB REPO ANALYZER — Full Report
  Generated: 29 March 2026, 14:32:10
  By: Krish Kumar | 24BAI10940
==============================================================

  ── REPOSITORY OVERVIEW ──────────────────────────────────
  Name          : pallets/flask
  Description   : The Python micro framework for building web apps
  License       : BSD 3-Clause License
  Topics        : flask, python, web, wsgi

  ── STATISTICS ────────────────────────────────────────────
  Stars         : 67,423
  Forks         : 16,891
  Open Issues   : 8
  Last Pushed   : 2 days ago

  ── LANGUAGE BREAKDOWN ────────────────────────────────────
  Python                ██████████████████████  97.3%
  Makefile              ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   1.4%

  ── TOP CONTRIBUTORS ──────────────────────────────────────
   1. davidism          ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  2,341 commits
   2. pallets-bot       ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓    891 commits

  ── QUICK SUMMARY ─────────────────────────────────────────
  Popularity    : Extremely popular (50k+ stars)
  Fork ratio    : 25.1% of stars have been forked
  Repo age      : 5,234 days (14 years)
==============================================================
```

09 HOW TO RUN IT

The tool requires only JDK 11 or higher — no additional packages or build tools.

```
# Step 1: Clone the repository
git clone https://github.com/krish0455/oss-audit-24bai10940.git
cd oss-audit-24bai10940/byop

# Step 2: Compile (one time)
javac GitHubAnalyzer.java

# Step 3: Run on any public GitHub repo
java GitHubAnalyzer https://github.com/python/cpython
java GitHubAnalyzer https://github.com/torvalds/linux
```

```
java GitHubAnalyzer https://github.com/microsoft/vscode

# Step 4: View search history
java GitHubAnalyzer --history

# Optional: Set token for higher rate limits
export GITHUB TOKEN=your personal access token
java GitHubAnalyzer https://github.com/pallets/flask
```

Reports are automatically saved to a reports/ folder in the same directory. Each filename includes the repo name and timestamp so you can build up a collection of analyses over time.

10 WHAT I LEARNED

The most significant learning from the Java port was the contrast in standard libraries. Python ships a JSON parser, a CSV writer, and clean HTTP support in its standard library. Java's JDK 11 provides an excellent HTTP client but no JSON parser at all. Writing one from scratch forced a deep understanding of how recursive descent parsing works — something that libraries normally hide entirely.

The type system difference was also instructive. Python's duck typing means a JSON object just becomes a dict with no ceremony. Java requires explicit casts from Object to Map or List, plus null checks at every level. The helper methods (str, map, list, toLong) in the Java port exist precisely to make this safe without constant try-catch blocks.

Error handling in Java is more explicit than Python. Every checked exception must be declared or caught. This is verbose but it makes the failure modes of every network call visible in the code, which is actually useful when building a tool that depends on an external API.

Finally, the zero-dependency constraint was more challenging in Java than in Python — but the result is the same: a tool that anyone can compile and run immediately, without a package manager, without version conflicts, and without anything beyond a standard JDK installation.

11 CONCLUSION

The GitHub Repo Analyzer started as a solution to something genuinely annoying — having to open four or five GitHub pages just to evaluate one repository. The final tool solves that with a single command, gives you more information than any single GitHub page shows, saves it for later, and keeps a history of everything you have looked at.

The Java port adds an additional dimension: it demonstrates that the same zero-dependency philosophy that worked in Python is achievable in Java too, just with different tradeoffs. The custom JSON parser is the most interesting artefact of that constraint — a piece of infrastructure that would normally come from a library, written from scratch to keep the tool self-contained.

Every part of the tool — from the URL parser to the CSV logger to the API error handling to the recursive-descent JSON parser — was a deliberate design decision. That is what purposeful and well-executed means, and that is what this project aimed to build.

Student Name	Krish Kumar
Registration	24BAI10940
Project	GitHub Repo Analyzer
Language	Java 11+

GitHub	https://github.com/krish0455/oss-audit-24bai10940
Tool File	byop/GitHubAnalyzer.java
Submission	VITyarthi Portal